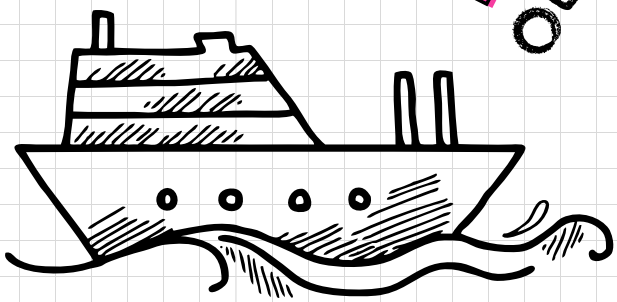
AMİRAL BATTI OYUNU!



DEFNE

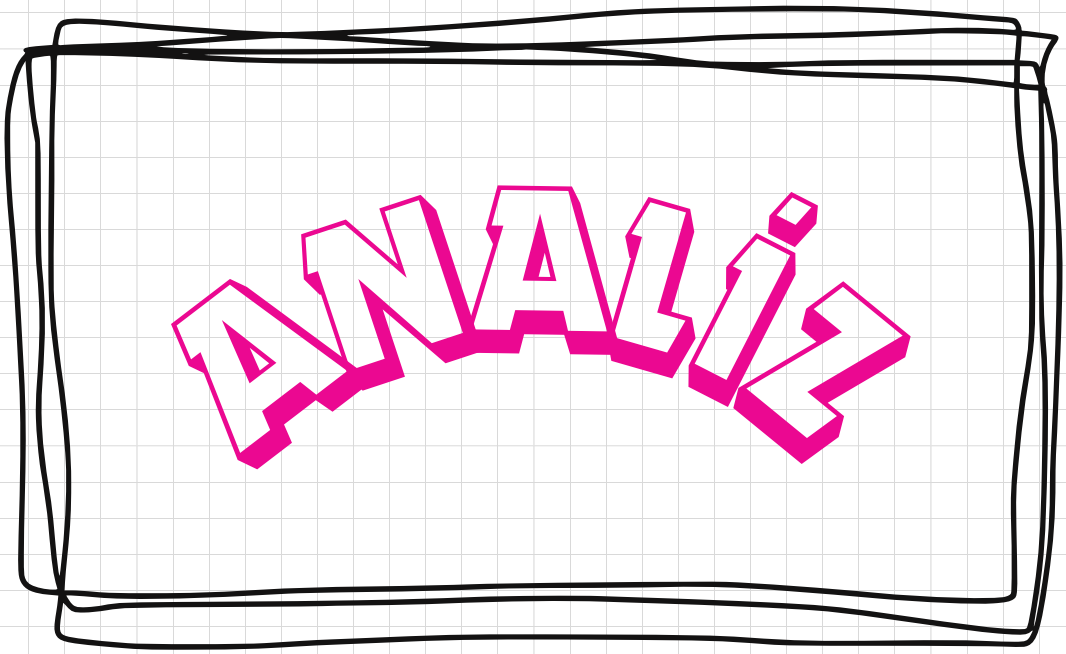
GÖDELEKOĞLU

220710071

GÖRSEL PROGRAMLAMA PROJESİ

İçindekiler:

1. Analiz
2. Tasarım
3. Programcı Kataloğu
4. Kullanıcı Kataloğu



PROJE ANALİZİ + PROBLEM TANIMI

GÖRSEL PROGRAMLAMA PROJESİ

Projenin temel problemi, kullanıcıya gemileri gizli bir şekilde yerleştirilmiş bir tahta üzerinde buldurmak ve bu gemilerin her bir parçasının 8 komşuluğuna diğer gemi parçalarının denk gelmemesini sağlamaktır. Aynı zamanda, kullanıcının bir gemiyi tamamen bulduğunda, o gemiye ait parçaların bulunduğu konumun belirgin bir şekilde işaretlenmesi ve geminin etrafındaki alanın kapatılması gerekmektedir.

Projenin başarıyla gerçekleştirilmesi için aşağıdaki temel gereksinimler belirlenmiştir:

- 15x15 boyutunda bir satranç tahtası oluşturulmalı.
- Belirli uzunluktaki gemiler (1 adet 4 birim, 2 adet 3 birim, 3 adet 2 birim, 4 adet 1 birim) rasgele ve çakışmamak üzere yerleştirilmelidir.
- Kullanıcıya gemilerin konumu gösterilmeden oyun başlatılmalıdır.
- Kullanıcı, her seferinde bir hücre seçerek gemileri bulmaya çalışmalıdır.
- Eğer bir hücre seçildiğinde gemi parçasına denk gelinirse, bu durum belirgin bir şekilde işaretlenmeli ve kullanıcıya geminin etrafındaki alanlar kapatılmalıdır.
- Tüm gemiler batırıldığında, kullanıcıya harcanan süre ve/veya hamle sayısı gösterilmelidir.
- Kullanıcı yeni oyun başlattığında, gemilerin yerleşimi değişmelidir.



PROGRAM PROTATİPİ

GÖRSEL PROGRAMLAMA PROJESİ

Programım, Amiral Battı adlı bir oyunu içermektedir. Bu oyun, 15x15 boyutlu bir satranç tahtası üzerinde rasgele yerleştirilen gemilerin, kullanıcı tarafından bulunması esasına dayanmaktadır.

Programın tasarımını oluştururken, birden fazla kavramın ifade edilmesi için ayrı sınıflar oluşturdum. Bu sınıfların veri elemanlarını private olarak tanımlayarak, çağrılan metotlarını ise public olarak belirledim. Tasarımımda önemli olan noktalardan biri, kodun modüler olması ve farklı sorumluluklara sahip sınıflar arasında net bir ayrım yapılmasıdır.

Oyun içerisinde gemi yerleştirme, zaman kontrolü, buton tıklama işlemleri gibi farklı görevleri modüller halinde ayırdım. Bu, kodun daha düzenli ve okunabilir olmasını sağlamaktadır.

Ayrıca, oyundaki gemilerin yerleştirilmesi için sıralı bir çift-bağlı liste kullandım. Her bir gemi sınıfı, geminin başlangıç koordinatları, uzunluğu, yerleştirilip yerleştirilemeyeceği gibi özelliklere sahiptir. Bu, gemi yerleştirme işlemlerinin daha düzenli ve sıralı bir şekilde gerçekleştirilmesini sağlar.

Oyunun işleyişini kontrol etmek için bir zamanlayıcı (timer) kullandım. Oyun başladığında timer başlar ve her saniye geçtikçe dakika ve saniye hesaplamaları yapılır. Bu sayede, kullanıcının oyunu ne kadar sürede tamamladığı ölçülerek en iyi süre kaydedilir.

Kod içerisinde, kullanıcıya bilgi vermek ve etkileşimi artırmak adına görsel dosyalar kullandım. Gemi bulunması durumunda geminin etrafını belirten bir görsel eklenmiş ve süre bazında farklı ifadeler içeren görseller kullanılmıştır.

Bu tasarım, kodun modüler ve okunabilir olmasını hedeflemekte ve kullanıcıya etkileşimli bir oyun deneyimi sunmayı amaçlamaktadır.

Kod yazım işlemi toplamda 21 gün içinde tamamlanmış ve projenin bitiş tarihi olarak 3 Ocak 2024 belirlenmiştir.

PROGRAMDA KULLANILAN KÜTÜPHANELER

Bu proje görseller,textler ve listeler üzerinde işlem yapmak için çeşitli kütüphaneler içermektedir.

GÖRSEL PROGRAMLAMA PROJESİ

1.System:

- Genel olarak C# dilinde temel sağlayıcı sınıfları içeren bir kütüphanedir. Bu kütüphane, temel veri türleri, dosya işlemleri, iş parçacığı yönetimi ve diğer çeşitli temel işlevleri içerir.
- Kodumun genel programlama yapısını ve temel veri türlerini kullanmak için bu kütüphaneyi ekledim.

2.System.Collections.Generic:

- Genel koleksiyon sınıflarını içerir. Örneğin, List, Dictionary gibi veri yapılarını kullanmamızı sağlar.
- “List<YildizGrubu>” gibi genel koleksiyonları kullanmak için ekledim. Kodumda gemi gruplarını içeren bir liste bulunmaktadır.

3.System.Drawing:

- Grafiksel işlemler için kullanılan bir kütüphanedir. Özellikle Image sınıfını içerir ve resim dosyalarını işlemek için kullanılır.
- Görsel işlemler yapabilmek için bu kütüphaneyi kullandım. Özellikle gemi veya vurulan alanların görsel temsili için Image sınıfını kullandım. (Image.FromFile, ResimBoyutunuAyarla fonksiyonları).

4.System.IO:

- Giriş/Çıkış işlemlerini yöneten sınıfları içerir. Dosya okuma/yazma, dizin yönetimi gibi işlemleri gerçekleştirmek için kullanılır.
- Dosya işlemleri yapabilmek için kullandım. Görsel dosyaların yollarını belirtmek ve kontrol etmek amacıyla.

5.System.Text:

- Metin işlemleri için kullanılan sınıfları içerir. String manipülasyonu ve encoding/decoding işlemleri gibi işlevleri sağlar.

6.System.Windows.Forms:

- Windows Forms uygulamaları geliştirmek için kullanılan bir kütüphanedir. Bu kütüphane, pencere ve kontrol oluşturma, olayları yönetme gibi görsel arayüz bileşenlerini içerir.(Form, Button, PictureBox gibi kontroller).

PROGRAMDA KULLANILAN METODLAR

Programda kullanılan metodlar kısmı, projenin temel işlevselliğini sağlayan ve çeşitli işlemleri gerçekleştiren fonksiyonları içerir.

GÖRSEL PROGRAMLAMA PROJESİ

```
public Form1()
{
    InitializeComponent();
    this.WindowState = FormWindowState.Maximized;
    TahtayiHazirla();
    Zaman = new System.Windows.Forms.Timer();
    Zaman.Interval = 1000;
    Zaman.Tick += Zamanlayici;
    Zaman.Start();
}
```

Bu kod parçası, bir Windows Forms uygulamasındaki bir formun (Form1) kurucu metodunu temsil eder.
InitializeComponent(): Bu metot, Windows Forms tasarım arayüzünde oluşturulan bileşenleri başlatmak ve formun diğer özelliklerini ayarlamak için otomatik olarak üretilen bir metottur.
this.WindowState = FormWindowState.Maximized: Bu satır, formun başlangıçta ekranın tamamını kaplamasını sağlar.
TahtayiHazirla(): Bu metot, TahtayiHazirla adlı başka bir metodu çağırır.
Zaman = new System.Windows.Forms.Timer(): Bu satırda bir Timer nesnesi oluşturulur.
Zaman.Interval = 1000: Timer'ın tetikleme aralığını 1000 milisaniye (1 saniye) olarak ayarlar.
Zaman.Tick += Zamanlayici: Timer'ın tetiklendiğinde çağrılacak olan olaya (Tick) bir olay işleyici ekler. Bu durumda Zamanlayici adlı metod tetiklenecektir.
Zaman.Start(): Timer'ı başlatır. Bu, timer'ın belirlenen aralıklarda tetiklenmeye başlamasını sağlar.

```
private void Form1_Load(object sender, EventArgs e)
{
    Zaman.Start();
}
```

Bu metodda formun başlatılması ve başlangıç durumunun ayarlanması gibi işlemler yapılır. Bu durumda, form yüklendiğinde Zaman adlı Timer başlatılmış olacak ve belirli aralıklarla tetiklenecek olan olaylar gerçekleştirilecektir.

```
private void button1_Click(object sender, EventArgs e)
{
    OyunuSifirla();
}
```

Bu metodda buton 1 e tıklandığında OyunuSİfirla isimli metod çağrılır.

```
private void Buton_MouseGiris(object sender, EventArgs e)
{
    Button üzerindeDurulanDüğme = (Button)sender;
    üzerindeDurulanDüğme.BackColor = Color.White;
}
```

Bu satırda, olayı tetikleyen nesneyi bir “Button” türüne dönüştürerek “üzerindeDurulanDüğme” adlı değişkene atar. “sender” parametresi genellikle object türündedir ve bu tür, daha spesifik bir türe dönüştürülerek kullanılır.
üzerindeDurulanDüğme.BackColor = Color.White: Bu satır, butonun arka plan rengini beyaz (White) olarak ayarlar. Yani, fare bu butonun üzerine geldiğinde, butonun arka plan rengini beyaz yapar.

```
private void Buton_MouseCikis(object sender, EventArgs e)
{
    Button ayrilanDüğme = (Button)sender;
    ayrilanDüğme.BackColor = Color.FromArgb(5, Color.Pink);
}
```

Bu satırda, olayı tetikleyen nesneyi bir “Button” türüne dönüştürerek “ayrilanDüğme” adlı değişkene atar. “sender” parametresi genellikle object türündedir ve bu tür, daha spesifik bir türe dönüştürülerek kullanılır. ayrilanDüğme.BackColor =Color.FromArgb(5,Color.Pink): Bu satır, butonun arka plan rengini pembe (Pink) olarak ayarlar. Ancak, Color.FromArgb(5, Color.Pink) ifadesi ile bir renk oluşturulmuş ve bu renk biraz saydamlık içermektedir (5 saydamlık derecesidir). Bu nedenle, butonun arka plan rengi kısmen şeffaf bir pembe olacaktır.

```
private Image ResimBoyutunuAyarla(Image resim, Size boyut)
{
    return new Bitmap(resim, boyut);
}
```

return new Bitmap(resim, boyut): Bu satırda, Bitmap sınıfının bir örneği oluşturulur ve bu örneğin inşa edicisine resim ve boyut parametreleri ile geçilen değerler iletilir. Yani, mevcut resim belirtilen boyuta ölçeklenmiş bir yeni Bitmap örneği olarak geri döndürülür.

```
private class YildizGrubu
{
    5 başvuru
    public int BaslangicX { get; set; }
    5 başvuru
    public int BaslangicY { get; set; }
    8 başvuru
    public int Uzunluk { get; set; }
    4 başvuru
    public bool YatayMi { get; set; }
    4 başvuru
    public int TiklananYildizlar { get; set; }
}
```

Bu kod parçası, bir iç içe sınıf (nested class) olan YildizGrubu sınıfını tanımlar. Bu sınıfın amacı, oyun tahtasında yerleştirilecek olan gemi gruplarını (yıldızları) temsil etmektir. İşlevselliği aşağıdaki özelliklerle açıklanabilir:

1. public int BaslangicX { get; set; }: Bu özellik, yıldız grubunun başlangıç noktasının X koordinatını temsil eder.
2. public int BaslangicY { get; set; }: Bu özellik, yıldız grubunun başlangıç noktasının Y koordinatını temsil eder.
3. public int Uzunluk { get; set; }: Bu özellik, yıldız grubunun uzunluğunu (kaç birimlik olduğunu) temsil eder.
4. public bool YatayMi { get; set; }: Bu özellik, yıldız grubunun yatay mı dikey mi konumlandırılacağını belirtir. true ise yatay, false ise dikey konumlandırma yapılacak demektir.
5. public int TiklananYildizlar { get; set; }: Bu özellik, gemi grubuna ait kaç yıldızın (birim) kullanıcının tarafından tıklandığını temsil eder. Bu sayı, geminin tüm birimlerinin tıklanması durumunda sıfırlanabilir.

```
private void Zamanlayici(object sender, EventArgs e)
{
    saniye++;

    if (saniye == 60)
    {
        dakika++;
        saniye = 0;
    }

    label1.Text = $"{dakika:D2}:{saniye:D2}";
}
```

saniye++: Her zamanlayıcı tetiklendiğinde, saniye değişkeni bir arttırılır. Bu, oyunun süresini saniye cinsinden temsil eden bir değişkenin güncellenmesini sağlar.

if (saniye == 60): Eğer saniye değeri 60'a eşitse, bu durumda bir dakika geçmiş demektir. Bu durumda, dakika değişkeni bir arttırılır ve saniye sıfırlanır.

label1.Text = $"{dakika:D2}:{saniye:D2}";: Oyun süresini gösteren bir etiket (label1) üzerinde, dakika ve saniye değerleri iki basamaklı olarak görüntülenir. D2 format belirleyici, sayıları iki basamaklı olarak göstermek için kullanılır.

```
private void SureKontrol()
{
    int mevcutSure = dakika * 60 + saniye;
    pictureBox9.Visible = true;
    if (mevcutSure < enIyiSure)
    {
        enIyiSure = mevcutSure;
        label2.Text = $"Tebrikler! Yeni en iyi süre = {dakika:D2}:{saniye:D2}";
        pictureBox9.SizeMode = PictureBoxSizeMode.Zoom;
        pictureBox9.Image = Image.FromFile(@"C:\Users\defne\Desktop\Görsel Progarmlama C#\GörselProgram
    }
    else
    {
        pictureBox9.SizeMode = PictureBoxSizeMode.Zoom;
        label2.Text = $"Tebrikler! Süre = {dakika:D2}:{saniye:D2}.Fakat en iyisi degildi.";
        pictureBox9.Image = Image.FromFile(@"C:\Users\defne\Desktop\Görsel Progarmlama C#\GörselProgram
    }
}
```

int mevcutSure = dakika * 60 + saniye;: Mevcut oyun süresi, dakika cinsinden dakika değişkeni ve saniye cinsinden saniye değişkeni kullanılarak hesaplanır. Toplam süreyi saniye cinsine dönüştürmek amacıyla dakikalar 60 ile çarpılır ve saniye ile toplanır.

pictureBox9.Visible = true;: Oyun süresini kontrol ettikten sonra, pictureBox9 adlı PictureBox nesnesini görünür hale getirir. Bu PictureBox, oyuncuya görsel geri bildirim sağlamak için kullanılır.

if (mevcutSure < enIyiSure): Eğer mevcut süre, en iyi süreden daha kısa ise:

- **enIyiSure = mevcutSure;: En iyi süreyi günceller.**
- **label2.Text = $"...":Kullanıcıya label2 aracılığıyla tebrik mesajı gönderilir.**
- **pictureBox9.Image = Image.FromFile(....png"): PictureBox'a yola ait resmi ekler.**

else: Eğer mevcut süre, en iyi süreden daha uzun veya eşitse:

- **label2.Text = $"...":Kullanıcıya label2 aracılığıyla tebrik mesajı gönderilir.**
- **pictureBox9.Image = Image.FromFile(....png"): PictureBox'a yola ait resmi ekler.**

```
private void TahtayiHazirla()
{
    butonlar = new Button[tahtaBoyutu, tahtaBoyutu];
    int butonBoyutu = 60;
    int baslangicX = 543;
    int baslangicY = 79;

    Random rastgele = new Random();
    butonTiklamaDurumu = new bool[tahtaBoyutu, tahtaBoyutu];
    for (int i = 0; i < tahtaBoyutu; i++)
    {
        for (int j = 0; j < tahtaBoyutu; j++)
        {
            butonlar[i, j] = new Button();
            butonlar[i, j].Width = butonBoyutu;
            butonlar[i, j].Height = butonBoyutu;
            butonlar[i, j].Left = baslangicX + j * butonBoyutu;
            butonlar[i, j].Top = baslangicY + i * butonBoyutu;
            butonlar[i, j].Tag = new Point(i, j);
            butonlar[i, j].Click += Buton_Tikla;
            butonlar[i, j].MouseEnter += Buton_MouseGiris;
            butonlar[i, j].MouseLeave += Buton_MouseCikis;

            butonlar[i, j].ForeColor = Color.Transparent;

            butonlar[i, j].FlatAppearance.BorderColor = Color.LightPink;
            butonlar[i, j].FlatAppearance.BorderSize = 5;
            butonTiklamaDurumu[i, j] = false;
            Controls.Add(butonlar[i, j]);
        }
    }

    YildizGruplariniYerlestir(rastgele, 4, 1);

    YildizGruplariniYerlestir(rastgele, 3, 2);

    YildizGruplariniYerlestir(rastgele, 2, 3);

    YildizGruplariniYerlestir(rastgele, 1, 4);
}
```

Oyun tahtasını temsil eden iki boyutlu bir Button dizisi olan butonlar dizisini oluşturur.Butonların boyutu ve ilk butonun x,y kordinatları belirtilir.

Rastgele sayılar üretmek için bir nesne oluşturulur.Oyun tahtasındaki butonların tıklanma durumlarını izleyen iki boyutlu bir bool dizisi oluşturulur.

İki katmanlı bir döngü ile oyun tahtasındaki her bir butonun özelliklerini belirler ve bu butonları oluşturur.

Buton oluşturulur.Butonun üzerine tıklandığında hangi konumda olduğu takip edilir.Sırasıyla metodlar çağırılır.Butonun metin rengi ve kenarlık kalınlığı ayarlanır.Butonun tıklanma durumu false yapılır.Oluşturulan buton form kontrol koleksiyonuna eklenir.

4 birimlik 1 adet,3 birimlik 2 adet,2 birimlik 3 adet,1 birimlik 4 adet yıldız grubunu rastgele konumlandırmak için YildizGruplariniYerlestir metodunu çağırır.

```
private void YildizGruplariniYerlestir(Random rastgele, int uzunluk, int adet)
{
    for (int k = 0; k < adet; k++)
    {
        int rastgeleX = rastgele.Next(0, tahtaBoyutu);
        int rastgeleY = rastgele.Next(0, tahtaBoyutu);
        bool yatayYerlesim = rastgele.Next(2) == 0;

        YildizGrubu yildizGrubu = new YildizGrubu
        {
            BaslangicX = rastgeleX,
            BaslangicY = rastgeleY,
            Uzunluk = uzunluk,
            YatayMi = yatayYerlesim,
            TiklananYildizlar = 0
        };

        yildizGruplari.Add(yildizGrubu);
        if (yatayYerlesim && YildizGrubunuYerlestirebilirMi(rastgeleX, rastgeleY, uzunluk, true))
        {
            for (int l = 0; l < uzunluk; l++)
            {
                butonlar[rastgeleX, rastgeleY + l].Text = "*";
            }

            toplamYildiz += uzunluk;
        }
        else if (!yatayYerlesim && YildizGrubunuYerlestirebilirMi(rastgeleX, rastgeleY, uzunluk, false))
        {
            for (int l = 0; l < uzunluk; l++)
            {
                butonlar[rastgeleX + l, rastgeleY].Text = "*";
            }

            toplamYildiz += uzunluk;
        }
        else
        {
            k--;
        }
    }
}
```

Belirtilen adet yıldız grubunu oluşturmak için döngü başlatılır.Rastgele bir x ve y konumu seçilir.Yatay ya da dikey olma durumu rastgele belirlenir.

Yeni bir YildizGrubu nesnesi oluşturulur ve özellikleri belirlenir (Başlangıç x ve y konumları, uzunluk, yatay yerleşim bilgisi ve tıklanan yıldızlar sayısı). Oluşturulan YildizGrubu nesnesi, oyun tahtasındaki tüm yıldız gruplarını takip etmek için bir liste olan yildizGruplari listesine eklenir.

Eğer yatay yerleşim seçildiyse ve yıldız grubu belirtilen konuma yerleştirilebiliyorsa:Yıldız grubunu oluşturan butonlara '*' karakterini eklemek için bir döngü başlatılır.Yıldız grubunu oluşturan butona '*' karakteri eklenir.Toplam yıldız sayısı güncellenir.

Eğer yatay yerleşim seçilmediyse ve yıldız grubu belirtilen konuma yerleştirilebiliyorsa: Yıldız grubunu oluşturan butonlara '*' karakterini eklemek için bir döngü başlatılır. Yıldız grubunu oluşturan butona '*' karakteri eklenir.Toplam yıldız sayısı güncellenir.

Yatay veya dikey yerleşim durumunda yıldız grubu belirtilen konuma yerleştirilemiyorsa: Yeniden deneme yapabilmek için döngü sayacı azaltılır.

GÖRSEL PROGRAMLAMA PROJESİ

```
private bool YildizGrubunuYerlestirebilirMi(int x, int y, int uzunluk, bool yatay)
{
    if (yatay)
    {
        for (int l = 0; l < uzunluk; l++)
        {
            if (y + l >= tahtaBoyutu || butonlar[x, y + l].Text == "*")
            {
                return false;
            }
        }
    }
    else
    {
        for (int l = 0; l < uzunluk; l++)
        {
            if (x + l >= tahtaBoyutu || butonlar[x + l, y].Text == "*")
            {
                return false;
            }
        }
    }

    for (int i = -1; i <= uzunluk; i++)
    {
        for (int j = -1; j <= 1; j++)
        {
            int currentX = yatay ? x : x + i;
            int currentY = yatay ? y + i : y;

            if (currentX >= 0 && currentX < tahtaBoyutu && currentY >= 0 && currentY <
                tahtaBoyutu && butonlar[currentX, currentY].Text == "*")
            {
                return false;
            }

            if (currentX >= 0 && currentX < tahtaBoyutu && currentY + j >= 0 && currentY + j <
                tahtaBoyutu && butonlar[currentX, currentY + j].Text == "*")
            {
                return false;
            }

            if (currentY >= 0 && currentY < tahtaBoyutu && currentX + j >= 0 && currentX + j <
                tahtaBoyutu && butonlar[currentX + j, currentY].Text == "*")
            {
                return false;
            }
        }
    }

    return true;
}
```

Yatay konumda ve belirtilen uzunluktaki yıldız grubunun her bir elemanını kontrol etmek için bir döngü başlatılır. Eğer belirtilen konum, tahta boyutunu aşıyorsa veya o konumda zaten bir yıldız varsa, bu durumda yıldız grubu yerleştirilemez ve false döner.

Dikey konumda ve belirtilen uzunluktaki yıldız grubunun her bir elemanını kontrol etmek için bir döngü başlatılır. Eğer belirtilen konum, tahta boyutunu aşıyorsa veya o konumda zaten bir yıldız varsa, bu durumda yıldız grubu yerleştirilemez ve false döner.

Yatay yerleşimde, X konumunu değiştirmek için bir döngü başlatılır. Dikey yerleşimde, Y konumunu değiştirmek için bir döngü başlatılır. X veya Y konumlarını değiştirmek için bir döngü daha başlatılır.

Eğer yerleşim yatay ise, mevcut X konumu belirlenir, aksi halde Y konumu belirlenir. Mevcut Y konumu belirlenir, aksi halde X konumu belirlenir.

Belirli bir konumda yıldız grubu yerleştirebilme kontrolünü gerçekleştirir. İki durumu kapsar: yatay yerleşim ve dikey yerleşim. Bu koordinatların üzerinde veya yanında zaten bir yıldız (*) varsa veya bu koordinatlardan birine komşu bir konumda bir yıldız varsa, bir yıldız grubunu bu konuma yerleştirmenin mümkün olmadığını belirtir.

```
private void CevreyiDoldur(YildizGrubu yildizGrubu)
{
    int baslangicX = yildizGrubu.BaslangicX;
    int baslangicY = yildizGrubu.BaslangicY;
    int uzunluk = yildizGrubu.Uzunluk;
    bool yatayMi = yildizGrubu.YatayMi;

    string imagePath = @"C:\Users\defne\Desktop\Görsel Progarmlama C#\GörselProgramlamaProjesi\AmiralBattiOyunu2\GörsellerVeSesler\Karalama.jpeg";

    if (!File.Exists(imagePath))
    {
        MessageBox.Show("Alternatif görsel dosyası bulunamadı!");
        return;
    }

    for (int i = 0; i < uzunluk; i++)
    {
        int currentX = yatayMi ? baslangicX : baslangicX + i;
        int currentY = yatayMi ? baslangicY + i : baslangicY;

        if (butonlar[currentX, currentY].Text == "*" && butonlar[currentX, currentY].Image == null)
        {
            butonlar[currentX, currentY].Image = ResimBoyutunuAyarla(Image.FromFile(imagePath),
                butonlar[currentX, currentY].Size);
        }

        if (currentY - 1 >= 0 && butonlar[currentX, currentY - 1].Text != "*" && butonlar[currentX, currentY - 1].Image == null)
        {
            butonlar[currentX, currentY - 1].Image = ResimBoyutunuAyarla(Image.FromFile(imagePath),
                butonlar[currentX, currentY - 1].Size);
        }

        if (currentY + 1 < tahtaBoyutu && butonlar[currentX, currentY + 1].Text != "*" && butonlar[currentX, currentY + 1].Image == null)
        {
            butonlar[currentX, currentY + 1].Image = ResimBoyutunuAyarla(Image.FromFile(imagePath),
                butonlar[currentX, currentY + 1].Size);
        }
        if (currentX - 1 >= 0 && butonlar[currentX - 1, currentY].Text != "*" && butonlar[currentX - 1, currentY].Image == null)
        {
            butonlar[currentX - 1, currentY].Image = ResimBoyutunuAyarla(Image.FromFile(imagePath),
                butonlar[currentX - 1, currentY].Size);
        }

        if (currentX + 1 < tahtaBoyutu && butonlar[currentX + 1, currentY].Text != "*" && butonlar[currentX + 1, currentY].Image == null)
        {
            butonlar[currentX + 1, currentY].Image = ResimBoyutunuAyarla(Image.FromFile(imagePath),
                butonlar[currentX + 1, currentY].Size);
        }

        if (currentX - 1 >= 0 && currentY - 1 >= 0 && butonlar[currentX - 1, currentY - 1].Text != "*" && butonlar[currentX - 1, currentY - 1].Ima
        {
            butonlar[currentX - 1, currentY - 1].Image = ResimBoyutunuAyarla(Image.FromFile(imagePath),
                butonlar[currentX - 1, currentY - 1].Size);
        }

        if (currentX + 1 < tahtaBoyutu && currentY + 1 < tahtaBoyutu && butonlar[currentX + 1, currentY + 1].Text != "*" && butonlar[currentX + 1,
        {
            butonlar[currentX + 1, currentY + 1].Image = ResimBoyutunuAyarla(Image.FromFile(imagePath),
                butonlar[currentX + 1, currentY + 1].Size);
        }

        if (currentX + 1 < tahtaBoyutu && currentY - 1 >= 0 && butonlar[currentX + 1, currentY - 1].Text != "*" && butonlar[currentX + 1, currentY
        {
            butonlar[currentX + 1, currentY - 1].Image = ResimBoyutunuAyarla(Image.FromFile(imagePath),
                butonlar[currentX + 1, currentY - 1].Size);
        }

        if (currentX - 1 >= 0 && currentY + 1 < tahtaBoyutu && butonlar[currentX - 1, currentY + 1].Text != "*" && butonlar[currentX - 1, currentY
        {
            butonlar[currentX - 1, currentY + 1].Image = ResimBoyutunuAyarla(Image.FromFile(imagePath),
                butonlar[currentX - 1, currentY + 1].Size);
        }
    }
}
```

Görsel yolunu kontrol eder ve böyle bir görsel yoksa hata mesajı verir.

Eğer buton üzerinde yıldız var ve üzerine daha önce görsel eklenmediyse, belirtilen görsel bu konuma eklenir.

Ayrıca, belirli bir konumun çevresindeki (üst, alt, sol, sağ, çapraz) konumlar da kontrol edilir ve eğer daha önce eklenmemişse bu konumlar üzerine de görsel eklenir.

```
private void Buton_Tikla(object sender, EventArgs e)
{
    Button tiklananButon = (Button)sender;
    Point butonPozisyonu = (Point)tiklananButon.Tag;

    if (butonTiklamaDurumu[butonPozisyonu.X, butonPozisyonu.Y])
    {
        return;
    }

    butonTiklamaDurumu[butonPozisyonu.X, butonPozisyonu.Y] = true;

    if (tiklananButon.Text == "*")
    {
        string imagePath = @"C:\Users\defne\Desktop\Görsel Progarmlama C#\GörselProgramlamaProjesi\AmiralBattiOyunu2\GörsellerVeSesler\Gemi.jpg";

        if (File.Exists(imagePath))
        {
            tiklananButon.Image = ResimBoyutunuAyarla(Image.FromFile(imagePath), tiklananButon.Size);

            tiklananYildizlar++;

            YildizGrubu bulunanGrup = null;
            foreach (var yildizGrubu in yildizGruplari)
            {
                if ((yildizGrubu.YatayMi && yildizGrubu.BaslangicX == butonPozisyonu.X && butonPozisyonu.Y >=
                    yildizGrubu.BaslangicY && butonPozisyonu.Y < yildizGrubu.BaslangicY + yildizGrubu.Uzunluk) ||
                    (!yildizGrubu.YatayMi && yildizGrubu.BaslangicY == butonPozisyonu.Y && butonPozisyonu.X >=
                    yildizGrubu.BaslangicX && butonPozisyonu.X < yildizGrubu.BaslangicX + yildizGrubu.Uzunluk))
                {
                    bulunanGrup = yildizGrubu;
                    break;
                }
            }

            if (bulunanGrup != null)
            {
            bulunanGrup.TiklananYildizlar++;

            if (bulunanGrup.TiklananYildizlar == bulunanGrup.Uzunluk)
            {
                if (bulunanGrup.Uzunluk == 4)
                {
                    pictureBox5.Visible = true;
                }
                else if (bulunanGrup.Uzunluk == 3)
                {
                    ucYildiz++;
                    if (ucYildiz == 2)
                        pictureBox4.Visible = true;
                }
                else if (bulunanGrup.Uzunluk == 2)
                {
                    ikiYildiz++;
                    if (ikiYildiz == 3)
                        pictureBox3.Visible = true;
                }
                else
                {
                    birYildiz++;
                    if (birYildiz == 4)
                        pictureBox2.Visible = true;
                }

                CevreyiDoldur(bulunanGrup);
                bulunanGrup.TiklananYildizlar = 0;
            }
        }

        if (tiklananYildizlar == toplamYildiz)
        {
            OyunTahtasiniGizle();
        }
    }
        else
        {
            MessageBox.Show("Görsel dosyası bulunamadı!");
        }
    }
    else
    {
        string alternativeImagePath = @"C:\Users\defne\Desktop\Görsel Progarmlama C#\GörselProgramlamaProjesi\AmiralBattiOyunu2\GörsellerVeSesler

        if (File.Exists(alternativeImagePath))
        {
            tiklananButon.Image = ResimBoyutunuAyarla(Image.FromFile(alternativeImagePath), tiklananButon.Size);
        }
        else
        {
            MessageBox.Show("Alternatif görsel dosyası bulunamadı!");
        }
    }
}
```

Eğer belirtilen konumda bir tıklama yapılmışsa metodun geri kalanını işlemeden doğrudan çıkar. Yani, aynı konumda birden fazla tıklamayı engeller.Eğer tıklama yapılmamışsa, bu konumdaki butonun tıklama durumu true olarak işaretlenir.

Tıklanan butonun hangi yıldız grubuna ait olduğunu belirler. Her yıldız grubu için, tıklanan butonun konumuyla yıldız grubunun konumu ve uzunluğunu karşılaştırarak bu grubu bulur. Eğer bir eşleşme bulunursa, bulunanGrup değişkenine bu yıldız grubunu atar ve döngüden çıkar.

Tıklanan yıldız grubunun kaç yıldız içerdiğini ve bu yıldız grubunun tamamının tıklanıp tıklanmadığını kontrol eder. Eğer tıklanan yıldız sayısı, yıldız grubunun uzunluğuna eşitse, o zaman bu yıldız grubu tamamen tıklanmış demektir. Bu duruma göre ilgili sayacı artırır ve bir PictureBox'ı görünür hale getirir. Ardından, CevreyiDoldur metodunu çağırarak, tıklanan yıldız grubunun çevresindeki butonlara görsel ekler ve tıklanan yıldız sayısını sıfırlar.

Tıklanan yıldızların sayısı toplam yıldıza eşitse yani bütün gruplar bulunduysa OyunTahtasiniGizle metodunu çağırır.

```
private void OyunTahtasiniGizle()
{
    Zaman.Stop();
    toplamYildiz = 0;
    tiklananYildizlar = 0;
    birYildiz = 0;
    ikiYildiz = 0;
    ucYildiz = 0;
    pictureBox1.Visible = false;
    pictureBox2.Visible = false;
    pictureBox3.Visible = false;
    pictureBox4.Visible = false;
    pictureBox5.Visible = false;
    pictureBox6.Visible = false;
    label1.Visible = false;
    for (int i = 0; i < tahtaBoyutu; i++)
    {
        for (int j = 0; j < tahtaBoyutu; j++)
        {
            butonlar[i, j].Visible = false;
        }
    }
    SureKontrol();
    label2.Visible = true;

    button1.Visible = true;
    pictureBox7.Visible = true;
    pictureBox8.Visible = true;
}
```

İlk olarak, oyun sırasında kullanılan bazı değişkenleri ve sayaçları sıfırlar. Ardından, PictureBox ve Label elemanlarını görünmez yapar. Son olarak, oyun tahtasındaki tüm butonları gizler ve ekranın alt kısmında bulunan bazı kontrol elemanlarını (button1, pictureBox7, pictureBox8) görünür hale getirir. Bu işlemlerle birlikte oyunun sona erdiği ve kullanıcıya sonuçları gösterdiği bir ekran oluşturulur.

```
private void OyunuSifirla()
{
    label2.Visible = false;
    button1.Visible = false;
    Zaman.Stop();
    dakika = 0;
    saniye = 0;
    pictureBox1.Visible = false;
    pictureBox2.Visible = false;
    pictureBox3.Visible = false;
    pictureBox4.Visible = false;
    pictureBox5.Visible = false;
    pictureBox6.Visible = false;
    pictureBox7.Visible = false;
    pictureBox8.Visible = false;
    pictureBox9.Visible = false;
    for (int i = 0; i < tahtaBoyutu; i++)
    {
        for (int j = 0; j < tahtaBoyutu; j++)
        {
            butonlar[i, j].Visible = false;
            butonlar[i, j].Image = null;
            butonTiklamaDurumu[i, j] = false;
        }
    }
    yildizGruplari.Clear();
    TahtayiHazirla();
    Zaman.Start();
    label1.Visible = true;
    pictureBox1.Visible = true;
    pictureBox6.Visible = true;
}
```

Oyun sırasında kullanılan bazı kontrol elemanlarını görünmez yapar ve oyun sırasında takip edilen süre ve sayaçları sıfırlar. Daha sonra, oyun tahtasındaki tüm butonları, butonlara ait resimleri ve buton tıklama durumlarını sıfırlar. Yıldız gruplarını temsil eden bir liste olan yildizGruplari listesini temizler ve yeni yıldız gruplarını yerleştirir. Son olarak, oyun süresini takip etmek için kullanılan timer (Zaman) tekrar başlatılır ve oyun ekranını gösteren bazı kontrol elemanları görünür hale getirilir. Bu işlemler, oyuncunun yeni bir oyun başlatmasına olanak tanır.

PROGRAMCI KATALOĞU

PROJE SÜRECİ + PROJE YÖNETİMİ

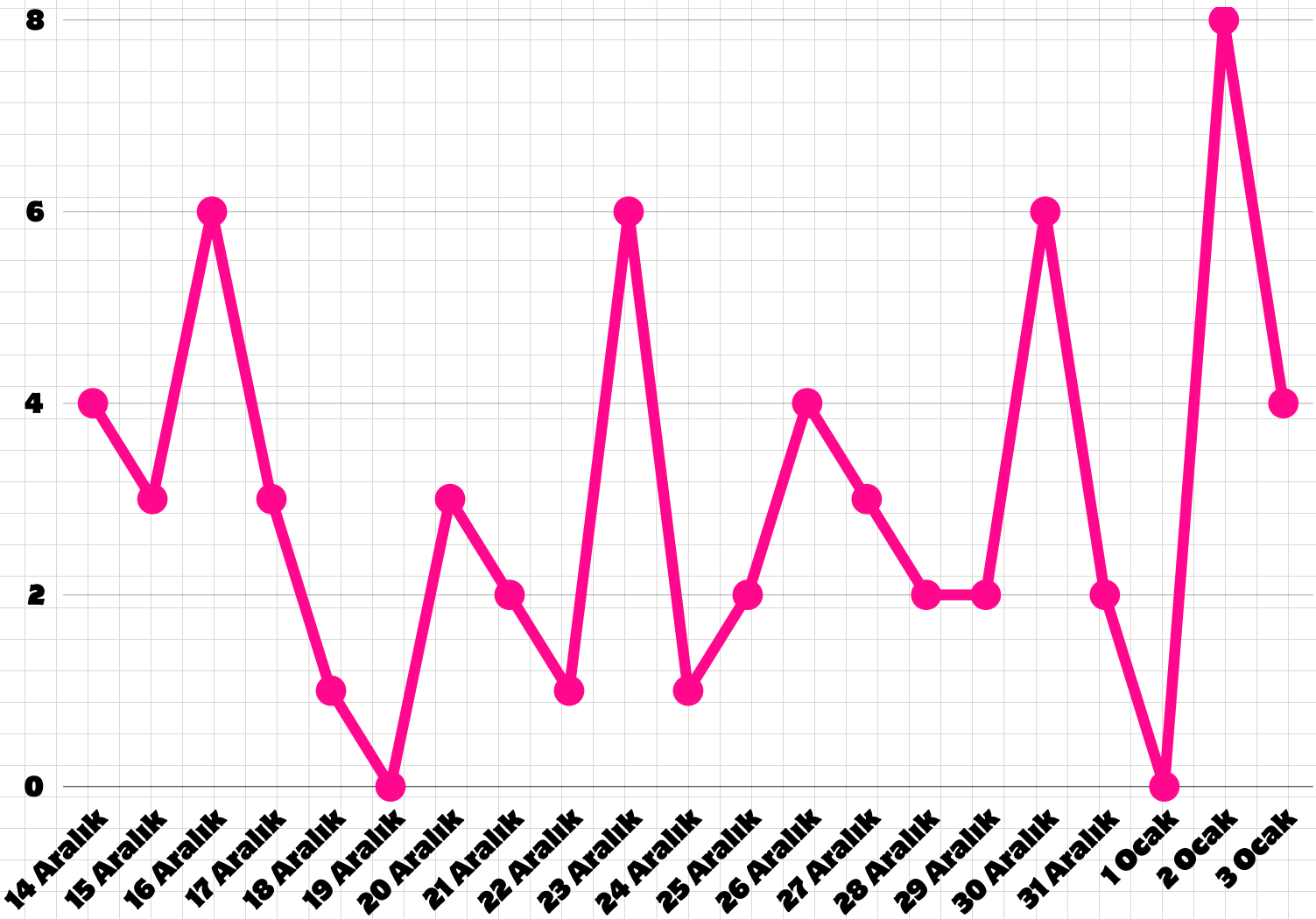
GÖRSEL PROGRAMLAMA PROJESİ

457
SATIR

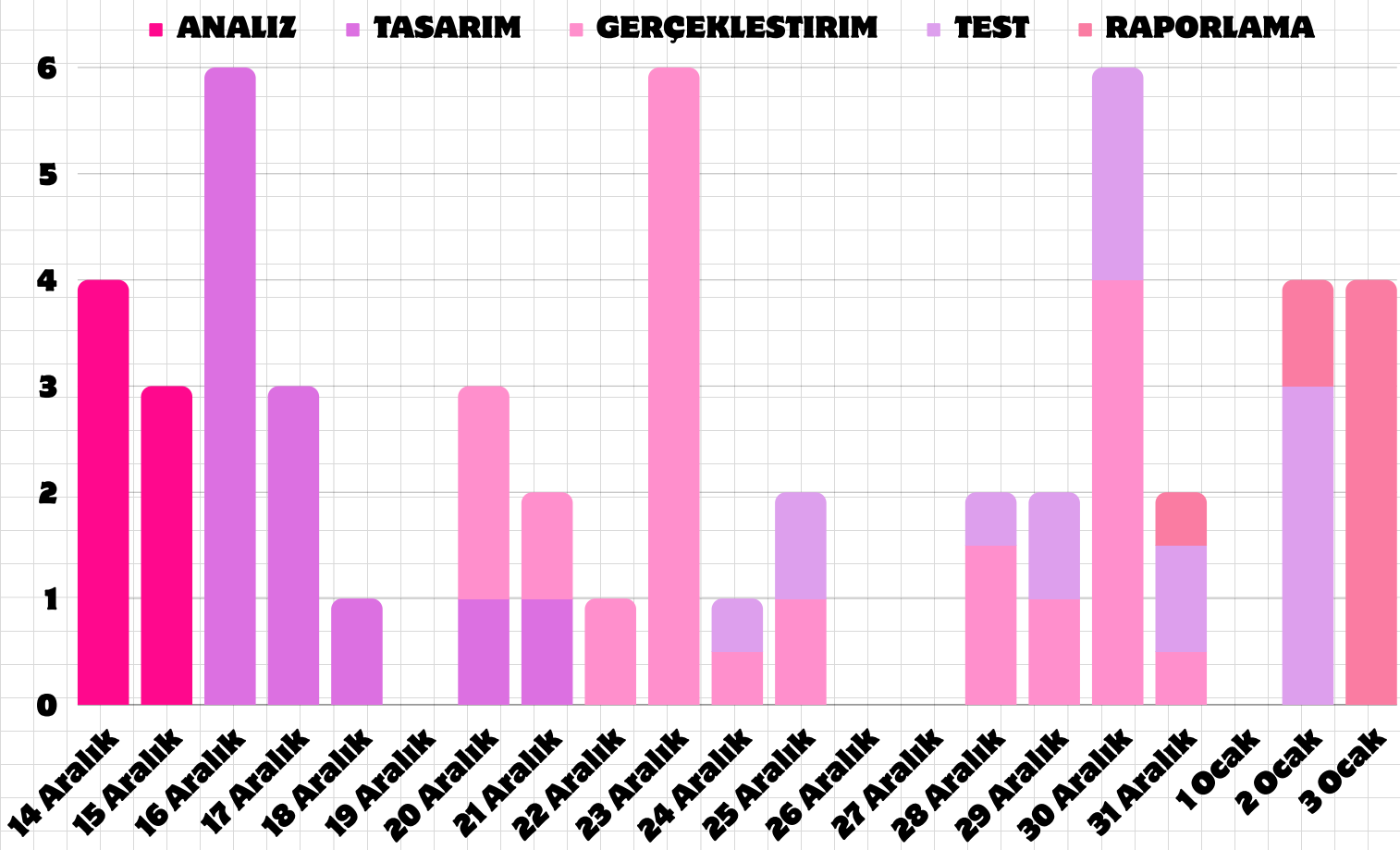
63
SAAT

21
GÜN

GÜNLÜK ÇALIŞMA SÜRELERİ



GÖRSEL PROGRAMLAMA PROJESİ



	1-2	3-4	5-6	7-8	9-10	11-12	13-14	15-16	17-18	19-20	21
ANALIZ											
TASARIM											
GERÇEKLESTIRIM											
TEST											
RAPORLAMA											

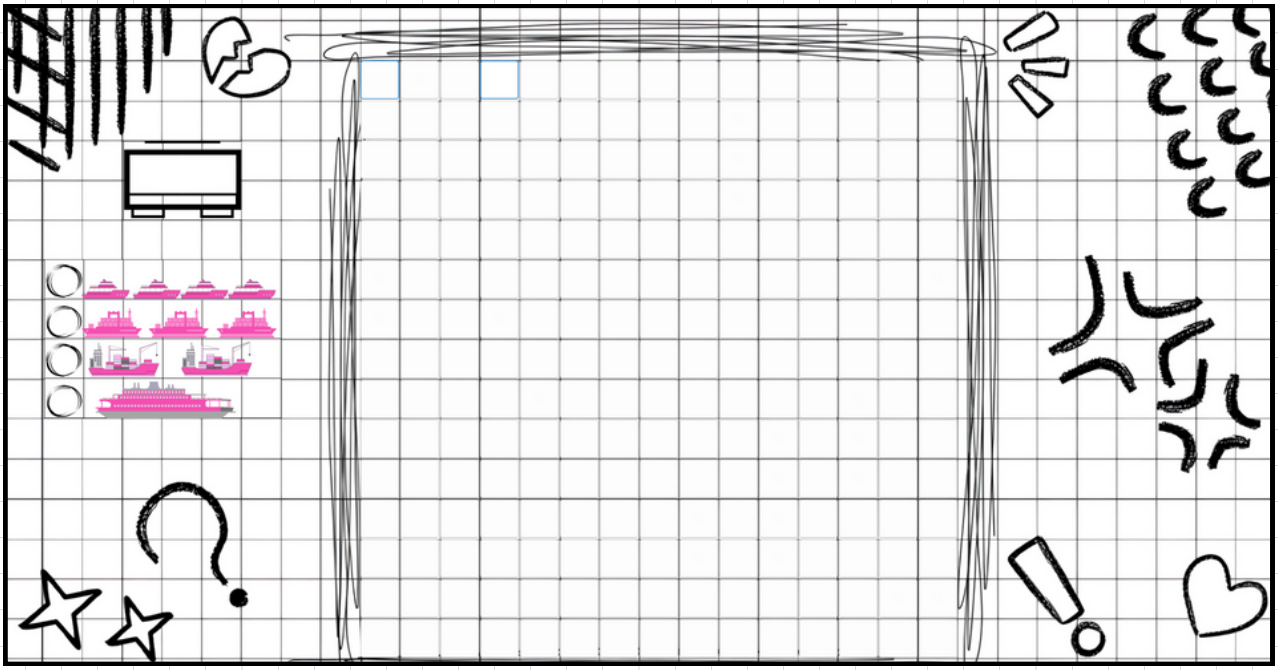
GÖRSEL PROGRAMLAMA PROJESİ

KULLANICI KATALOĞU

KULLANIM KILAVUZU + KISITLAMALAR

GÖRSEL PROGRAMLAMA PROJESİ

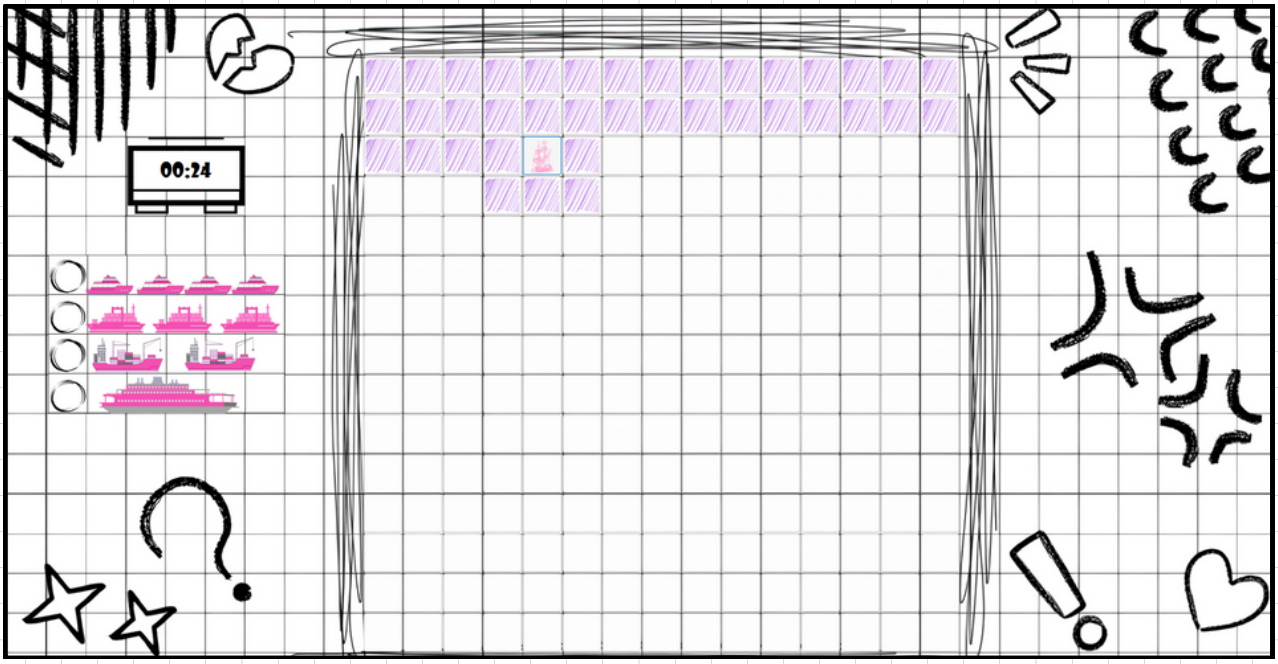
KULLANIM KILAVUZU



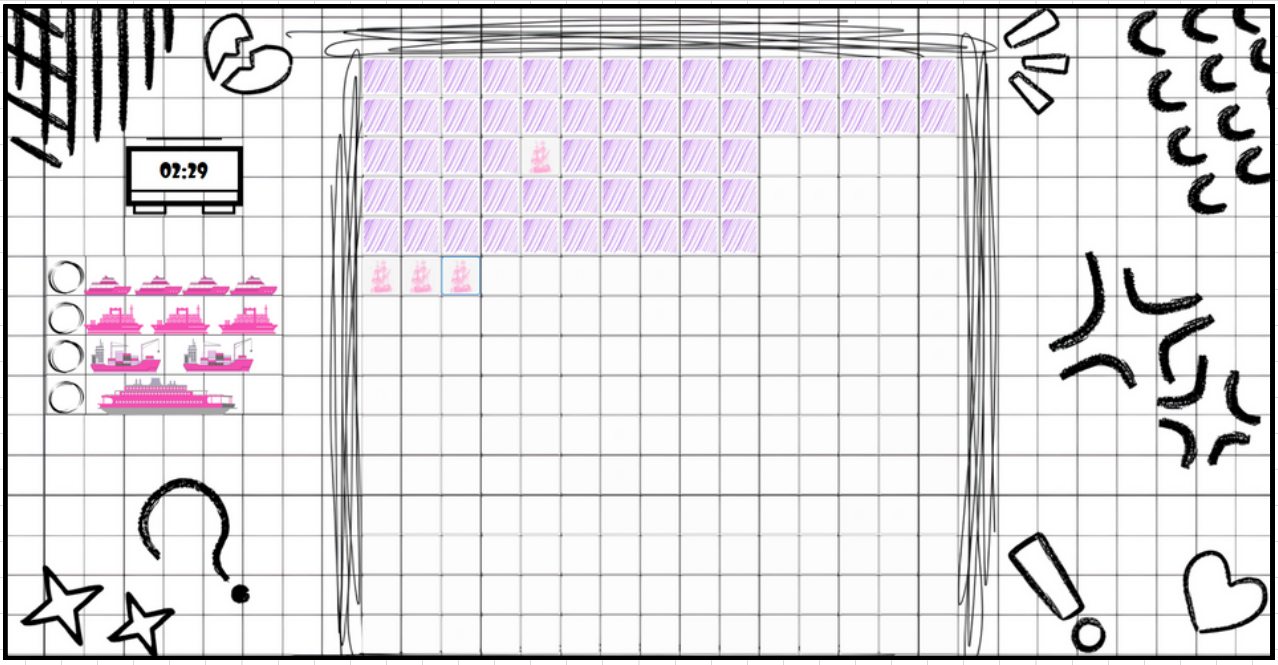
Oyun başlatıldığında Oyun Ekranı bizi karşılıyor.Oyun tahtası kullanıcın bulması için gemiler rastgele bir şekilde yerleştirilmiş olarak geliyor.

Form1 ekranı maximumda ekranı kaplıyor.

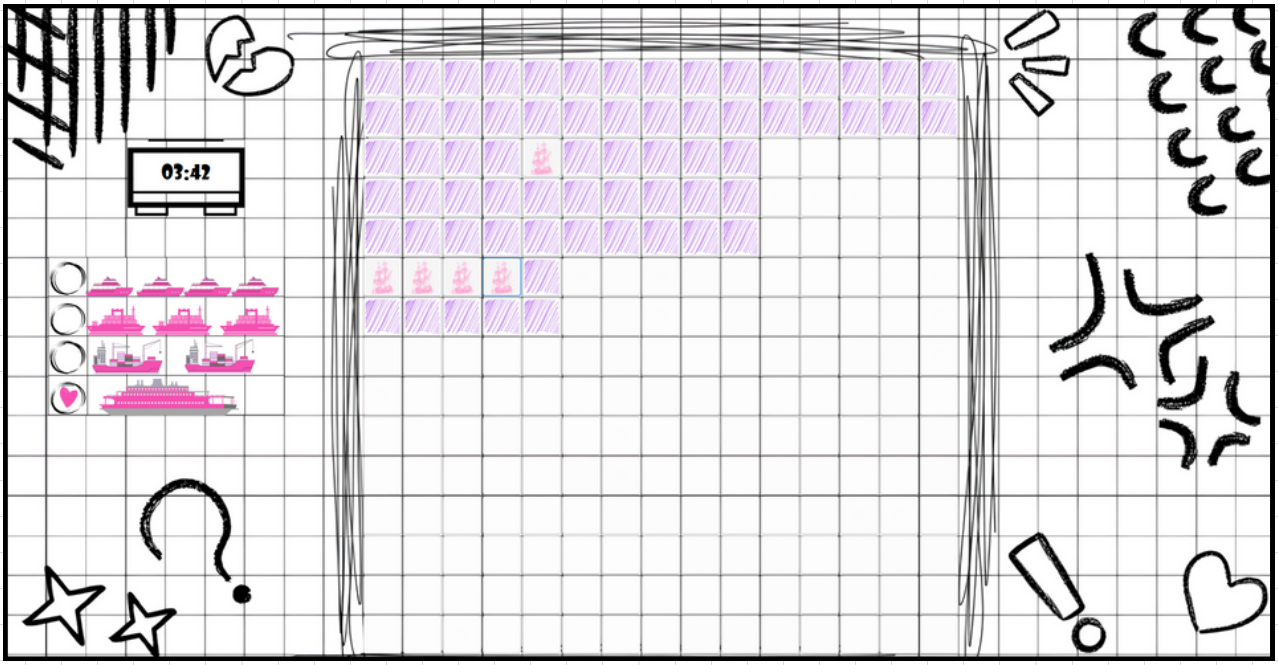
GÖRSEL PROGRAMLAMA PROJESİ



Kullanıcı her butona bastığında yapılan kontrol sayesinde butonda bir gemi veya geminin bir parçası varsa gemi görseli,buton boş ise karalama görseli butonu kaplıyor.

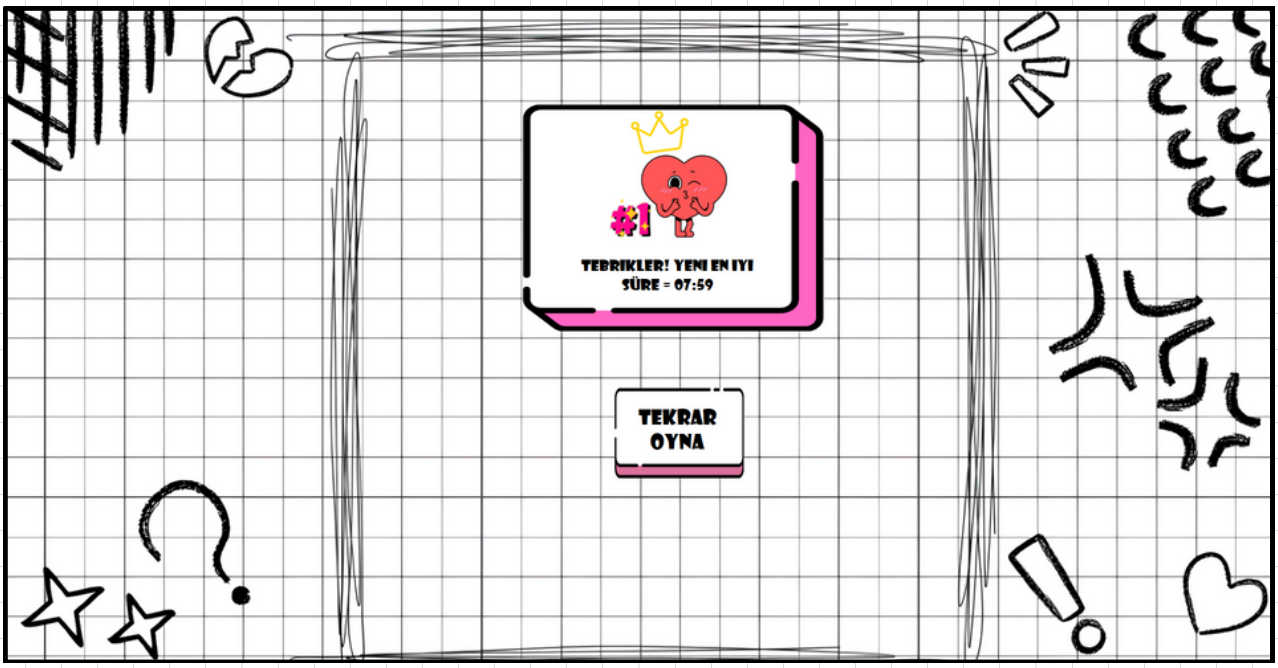


Geminin tamamı bulunana kadar geminin etrafındaki butonlar kapatılmıyor.



Geminin son parçası da bulunduğunda etrafında başka bir gemi bulunamayacağı için çevresi karalama görseli ile kapatılıyor.

Gemi uzunluğu aynı olan yani 1 adet 4 birim,2 adet 3 birim, 3 adet 2 birim ve 4 adet 1 birim uzunluğundaki gemilerden uzunluğu aynı olanların tamamı bulunduysa ilgili geminin yanına kalp işareti yerleşiyor.

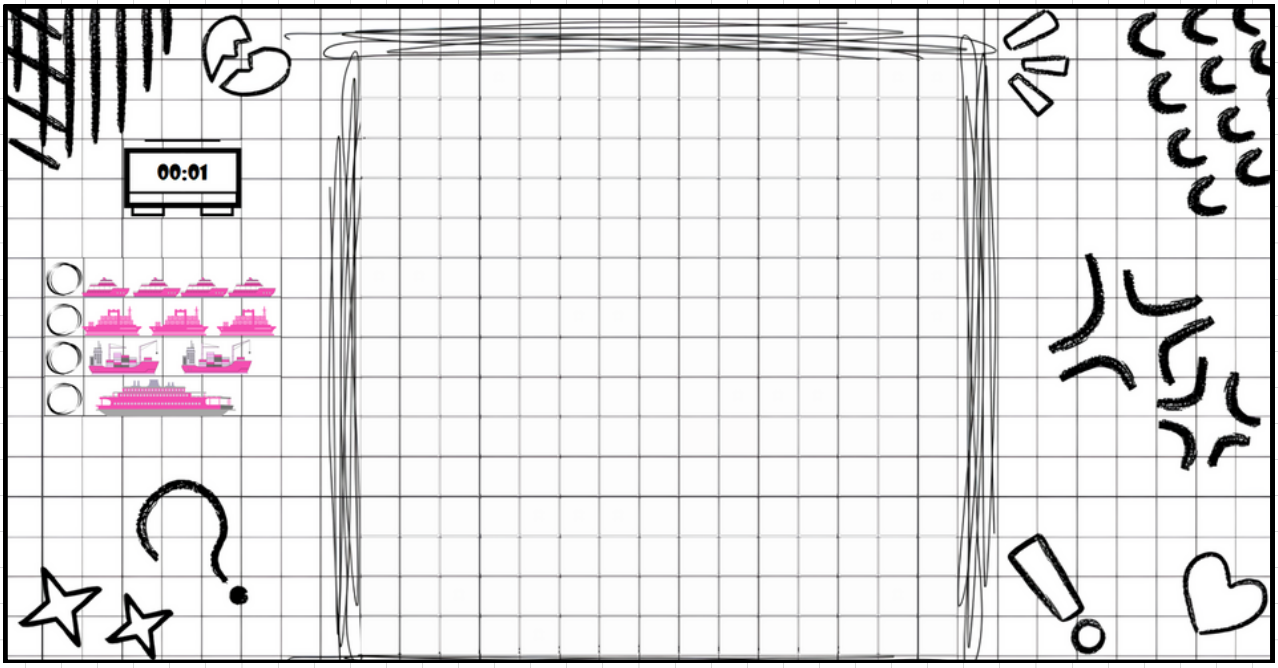


Bütün gemiler bulunduktan sonra bitiş ekranı yansıyor.

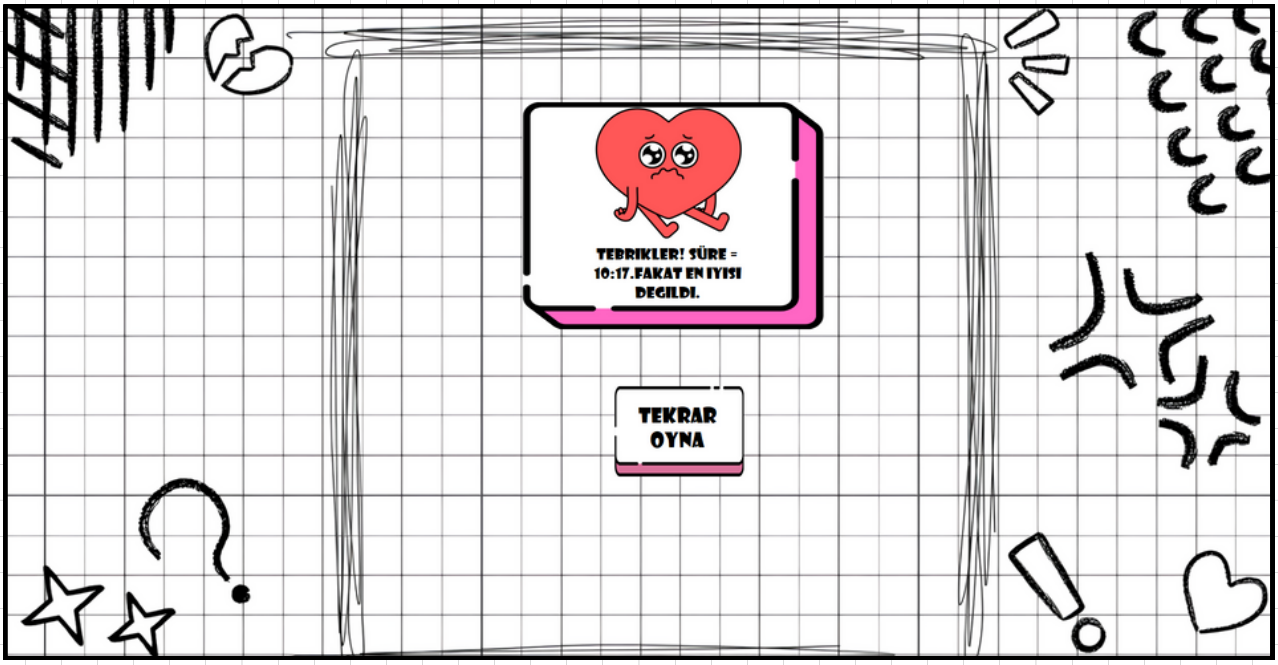
Ekrana oyunun ne kadar sürede bitirildiğine dair bilgi veriliyor.

İlk oynayıştaki süre her zaman en iyi süre kabul ediliyor bundan sonrakiler kendi aralarında karşılaştırılıp en iyi süreye karar veriliyor.

Tekrar oyna butonu bitiş ekranında aktif oluyor.



Tekrar oyna butonuna basıldığında bütün gemiler tekrar ve bir öncekinden farklı olarak dağılmış şekilse oyun yenileniyor.

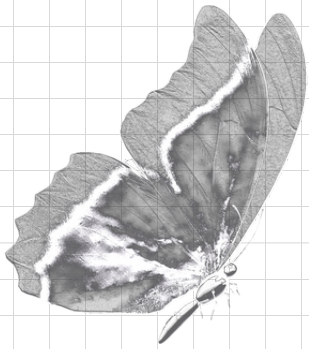


Kullanıcı tekrar oynayıp daha önceki en iyi süresinden daha kısa bir sürede bitiremezse ekrana süre bilgisi ve en iyi süre olmadığına dair bir mesaj gelir.

KISITLAMALAR

Oyunumdaki kısıtlamalar: herhangi bir gemi bulunduğunda o gemiyle aynı uzunluktaki bütün gemiler bulunmadığı sürece kullanıcıya geri dönüş yapmamaktadır.

Bir butona birden çok basılması durumunda işlem yapmaz fakat uyarı mesajı da vermez.



teşekkürler

Bu proje ödevini tamamlamak için bireysel olarak üzerinde çalıştığım süreçte, destek ve ilham alabileceğim birçok kaynağa sahip oldum. Özellikle, hocam Dr. Öğretim Üyesi Şeyma YÜCEL ALTAY'ın rehberliği ve öğretileri benim için büyük bir değer taşıdı. Farklı beceriler ve perspektifler kulllanarak projeyi başarıyla tamamlamak benim için önemli bir deneyimdi.